

LeetCode 3255: Find the Power of K-Size Subarrays II

Problem Analysis & Solutions Notebook

Problem Statement

For every window of size k , we need to return the maximum number **PROVIDED THAT** the numbers in the window are *increasing* and *consecutive*. Otherwise, return -1 .

Example: `arr = [1, 2, 3, 4, 7]`, $k = 3$

- `[1, 2, 3]` → Valid (Increasing & Consecutive). Return **3**.
- `[2, 3, 4]` → Valid (Increasing & Consecutive). Return **4**.
- `[3, 4, 7]` → Invalid (4 and 7 are not consecutive). Return **-1**.

1. Brute Force Approach

Strategy:

- Create a temporary queue where you push only the elements of the current window.
- Create a check function which processes the temporary queue.
- In case the constraints are satisfied, push the last element of the window into the answer array (as it's guaranteed to be the largest). Else, push -1 .

Time Complexity: $O(N \times K)$

(Checking n windows of size k)

Space Complexity: $O(K)$

(For temporary queue & vector)

Implementation

```
class Solution {
public:
    bool check(queue<int> temp){
        vector<int> arr;
        while(!temp.empty()){
            arr.push_back(temp.front());
            temp.pop();
        }
        int n=arr.size();

        for(int i=0;i<n-1;i++){
            if(arr[i+1]!=arr[i]+1){
                return false;
            }
        }
        return true;
    }

    vector<int> resultsArray(vector<int>& nums, int k) {
        int n=nums.size();
        if(k>n) return {};
        vector<int> ans;
        queue<int> temp;

        for(int i=0;i<k;i++){
            temp.push(nums[i]);
        }

        if(check(temp)) ans.push_back(nums[k-1]);
        else ans.push_back(-1);

        for(int j=k;j<n;j++){
            temp.pop();
            temp.push(nums[j]);
            if(check(temp)) ans.push_back(nums[j]);
            else ans.push_back(-1);
        }

        return ans;
    }
};
```

2. Optimal Approach

Strategy:

Maintain a streak variable to track the length of the current strictly increasing consecutive sequence.

Example Flow: arr = [1, 2, 3, 5, 6, 7, 8]

- 1 → streak: 1
- 2 → streak: 2
- 3 → streak: 3
- 5 → streak: 1 (Not consecutive from 3 to 5, reset streak)
- 6 → streak: 2

At every index, if **streak $\geq k$** , then it's a valid window. Hence, push the maximum element (which is the current element). Otherwise, push **-1**.

Time Complexity: **$O(N)$**

(Every element processed once)

Space Complexity: **$O(1)$**

(Only streak variable is maintained)

Implementation

```
class Solution {
public:
    vector<int> resultsArray(vector<int>& nums, int k) {
        int n=nums.size();
        if(k>n) return {};
        vector<int> ans;
        int streak=1;

        for(int i=0;i<n;i++){
            if(i>0 && nums[i]==nums[i-1]+1){
                streak++;
            }
            else{
                streak=1;
            }

            if(i>=k-1){
                if(streak>=k){
                    ans.push_back(nums[i]);
                }
                else{
                    ans.push_back(-1);
                }
            }
        }

        return ans;
    }
};
```